

UCS503 - Software Engineering

NexusLB

A Custom Reverse Proxy & Intelligent Load Balancer

Submitted by:

1024030313 Navjot Singh
1024030316 Shaina Gera
1024030320 Prabhgun Kaur

Group: 3C23 BE Third Year, CSE

Submitted to:

Jhonsy Bansal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	3
2	Methodology and Design	4
2.1	Software Development Life Cycle (SDLC)	4
2.2	System Architecture and Requirements	5
2.2.1	Functional and Non-Functional Requirements	5
2.2.2	Use Case Analysis	6
2.2.3	High-Level UML Architecture	7
2.2.4	Technical Stack Justification	7
2.3	Detailed Design and Implementation	9
2.3.1	Object-Oriented Design and Data Structures	9
2.3.2	Core Request Workflow and Activity Model	10
2.3.3	Load Balancing Strategies	11
2.3.4	Dual-Mode Health Checking and Failover	11
3	Results and Evaluation	12
3.1	Testing Methodology and Experimental Setup	12
3.2	Performance Metrics and Benchmark Results	12
3.3	Fault Injection and Self-Healing Evaluation	13
3.4	Engineering Challenges and Resolutions	14
3.4.1	Uneven Traffic in Least Connections Routing	14
3.4.2	Broken Sticky Sessions in IP Hash Balancing	14
3.4.3	Goroutine Leaks and Connection Drops in the Live Dashboard	15
4	Conclusion	16
4.1	Summary of Work	16
4.2	Key Findings	16
4.3	Future Work and Recommendations	16
4.4	Final Remarks	17
	Bibliography	18

Introduction

1.1 Background and Context

Modern software engineering is increasingly characterized by distributed, cloud-native architectures where client applications communicate with clusters of upstream microservices [1]. In such environments, incoming HTTP traffic is inherently bursty, characterized by volatile request volumes and non-uniform processing complexities. Simply provisioning multiple application backend servers behind independent network ports does not resolve the challenge of effective traffic distribution, nor does it guarantee high availability [7].

Without an intelligent intermediary layer, backend servers are vulnerable to uneven load distribution, where certain nodes experience resource saturation and queue buildup while others remain underutilized. Furthermore, when upstream instances crash due to memory leaks, hardware faults, or network partitions, naive clients continue attempting connections to dead endpoints. This leads to connection timeouts, socket resets, degraded user experience, and potential cascading outages across interconnected services [4].

Application-layer (Layer 7) reverse proxies and load balancers serve as the foundational ingress gateway in distributed infrastructure [6, 5]. By acting as a single, authoritative point of ingress, a reverse proxy terminates client TCP sessions, normalizes and enriches HTTP request headers (such as `X-Forwarded-For` and `X-Real-IP`), actively evaluates server operational health, and intelligently routes requests to healthy backend instances according to deterministic or load-sensitive balancing policies.

1.1.1 Problem Statement

While battle-tested enterprise load balancers such as NGINX and HAProxy dominate commercial production environments, their operational mechanics, concurrency synchronizations, and internal failure handlers often function as complex black boxes. Designing and constructing a custom, health-aware reverse proxy and load balancer from fundamental software engineering principles presents critical systems challenges:

1. **Inequitable Traffic Distribution:** Fixed routing schemes frequently fail to account for differing backend capacities and request runtimes, inducing latency spikes and localized server throttling.
2. **Dual-Mode Health Awareness:** Relying solely on passive error detection leaves client requests vulnerable to socket drops, while naive active health polling introduces detection latencies during high-frequency traffic windows. A robust system requires both proactive polling and reactive in-flight interception.
3. **High-Concurrency Synchronization Bottlenecks:** Coordinating dynamic backend registries and atomic metrics across thousands of concurrent goroutines necessitates non-blocking primitives to avoid thread contention, slice mutation races, and deadlocks.
4. **Zero-Downtime Rebalancing and Reintegration:** Upstream nodes must be

dynamically isolated upon failure and restored upon recovery without dropping active transactions or requiring gateway restarts.

5. **Real-Time Observability and Fault Injection:** System administrators require instantaneous, visual insight into live throughput, latency percentiles, and error rates, coupled with interactive failure simulation tools to evaluate cluster resilience under stress.

1.2 Project Objectives

The primary objective of **NexusLB** is to design, implement, and rigorously evaluate an intelligent, health-aware reverse proxy and load balancer in Go. Adhering to the SMART (Specific, Measurable, Achievable, Relevant, Time-bound) engineering framework, the project delivers:

- **Specific:** Construct a complete Layer 7 reverse proxy supporting configurable scheduling policies (Round-Robin, Least Connections, Deterministic IP Hash), an active/reactive health monitor, and an embedded live observability dashboard.
- **Measurable:** Achieve sub-millisecond routing latency overhead, 100% request continuity during single-node failure scenarios via reactive failover, zero data races under Go's race detector, and live telemetry updates delivered at 1 Hz over Server-Sent Events (SSE) [9].
- **Achievable:** Develop the core engine in Go using native standard library packages (`net/http`, `net/http/httputil`, `sync/atomic`) without heavy third-party framework dependencies, ensuring high portability and clean architectural boundaries [3].
- **Relevant:** Address foundational distributed systems concerns including atomic state synchronization, circuit isolation, and real-time observability.
- **Time-bound:** Deliver a verified, fully documented prototype accompanied by empirical performance benchmarks and interactive fault simulation by the Mid-Semester Evaluation milestone.

Methodology and Design

2.1 Software Development Life Cycle (SDLC)

NexusLB was developed using an **Agile Incremental Iterative Prototyping Model** [1]. Because concurrent network applications can manifest subtle timing vulnerabilities and memory race conditions, a monolithic development strategy would make debugging exceedingly difficult.

The project lifecycle was partitioned into five consecutive engineering iterations (Sprints). Each sprint produced a verified, runnable increment that was subjected to automated unit tests, concurrency race audits (`go test -race`), and structural code reviews before advancing to the next phase. Figure 2.1 illustrates the structured progression of the NexusLB SDLC, highlighting the feedback mechanisms between implementation, testing, and architectural refinement.

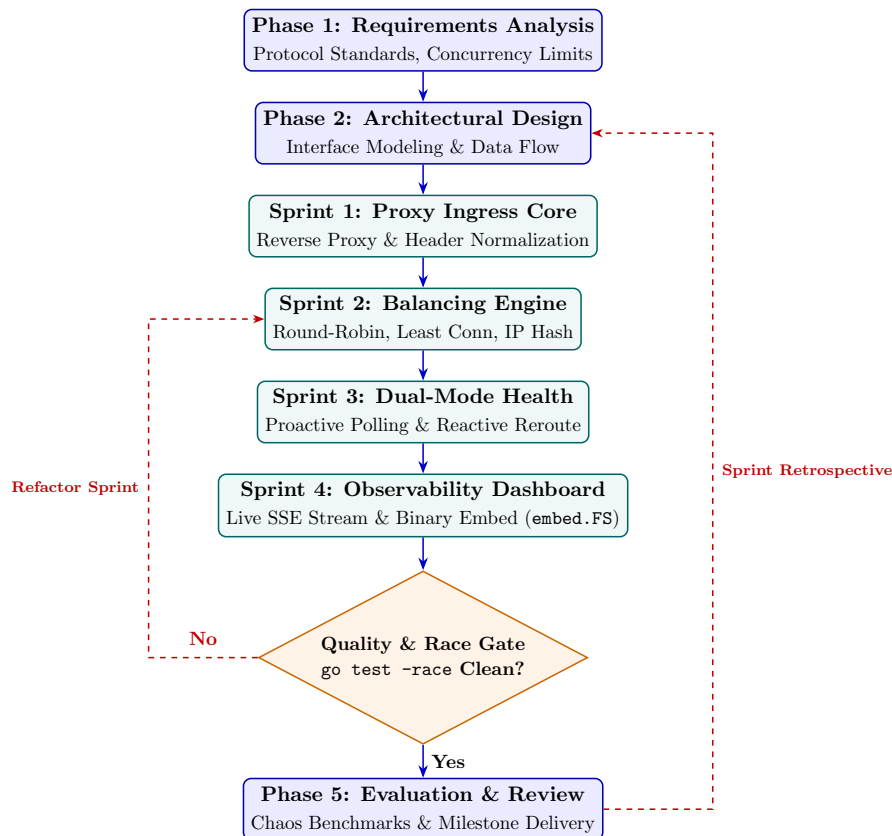


Figure 2.1: Agile Incremental Iterative SDLC model applied throughout the NexusLB project.

The incremental delivery schedule ensured structured milestones:

1. **Sprint 1 (Reverse Proxy Core):** Established the fundamental HTTP ingress pipeline on port 8080, header sanitation, client connection handling, and JSON configuration parsing.
2. **Sprint 2 (Pluggable Balancer Engine):** Designed the extensible `Balancer` interface and implemented the Round-Robin, Least Connections, and Deterministic IP Hash routing algorithms.
3. **Sprint 3 (Reliability & Failover):** Engineered proactive background health checks alongside in-flight reactive socket error catching to prevent dropped transactions during sudden upstream failures.
4. **Sprint 4 (Embedded Observability):** Developed the single-binary web dashboard on port 8081, utilizing Server-Sent Events (SSE) for live 1 Hz telemetry push and Go's `embed.FS` for compile-time asset embedding.
5. **Sprint 5 (Chaos Verification & Benchmarking):** Validated the system using simulated backend clusters (:8001, :8002, :8003), resolved slice mutation race conditions, and performed throughput/latency benchmarks.

2.2 System Architecture and Requirements

2.2.1 Functional and Non-Functional Requirements

The architectural design of NexusLB is established upon concrete functional and non-functional specifications:

- **Functional Requirements (FR):**

- *FR1 (Ingress Proxying):* Listen for incoming client HTTP connections on port 8080 and forward requests to designated upstream instances.
- *FR2 (Multi-Algorithm Dispatching):* Dynamically distribute requests across active backends using Round-Robin, Least Connections, or IP Hash algorithms.
- *FR3 (Proactive Health Checking):* Periodically query upstream `/health` endpoints to verify operational state and isolate unresponsive servers.
- *FR4 (Reactive In-Flight Interception):* Detect socket drops and connection resets mid-request, immediately flag the destination backend dead, and transparently reroute the request to an alternate healthy node.
- *FR5 (Live Observability Stream):* Transmit real-time telemetry (throughput, latency, active connections, error rate) via Server-Sent Events on port 8081.
- *FR6 (Interactive Chaos Simulation):* Provide administrator controls to simulate node crashes and observe dynamic rebalancing in real time.

- **Non-Functional Requirements (NFR):**

- *NFR1 (High-Concurrency Thread Safety):* Process thousands of simultaneous requests without data races, lock starvation, or pointer invalidations.
- *NFR2 (Minimal Dispatch Latency):* Ingress proxy routing overhead must not exceed 1 millisecond per transaction.
- *NFR3 (Zero External Runtime Dependencies):* The binary must be fully self-contained, embedding all web dashboard assets at compile time.
- *NFR4 (Resilient Error Handling):* Return structured HTTP 503 (Service

Unavailable) responses when all upstream backends are offline, preventing connection leaks.

2.2.2 Use Case Analysis

The system defines interactions with three primary actors:

1. **Client / Consumer:** Generates HTTP requests requiring fast, reliable responses.
2. **DevOps / System Administrator:** Monitors system performance, switches routing strategies, and triggers fault simulations.
3. **Upstream Backend Node:** Executes business logic and responds to health verification probes.

Figure 2.2 depicts the UML Use Case Diagram for NexusLB, illustrating system boundaries, primary use cases, and associative dependencies.

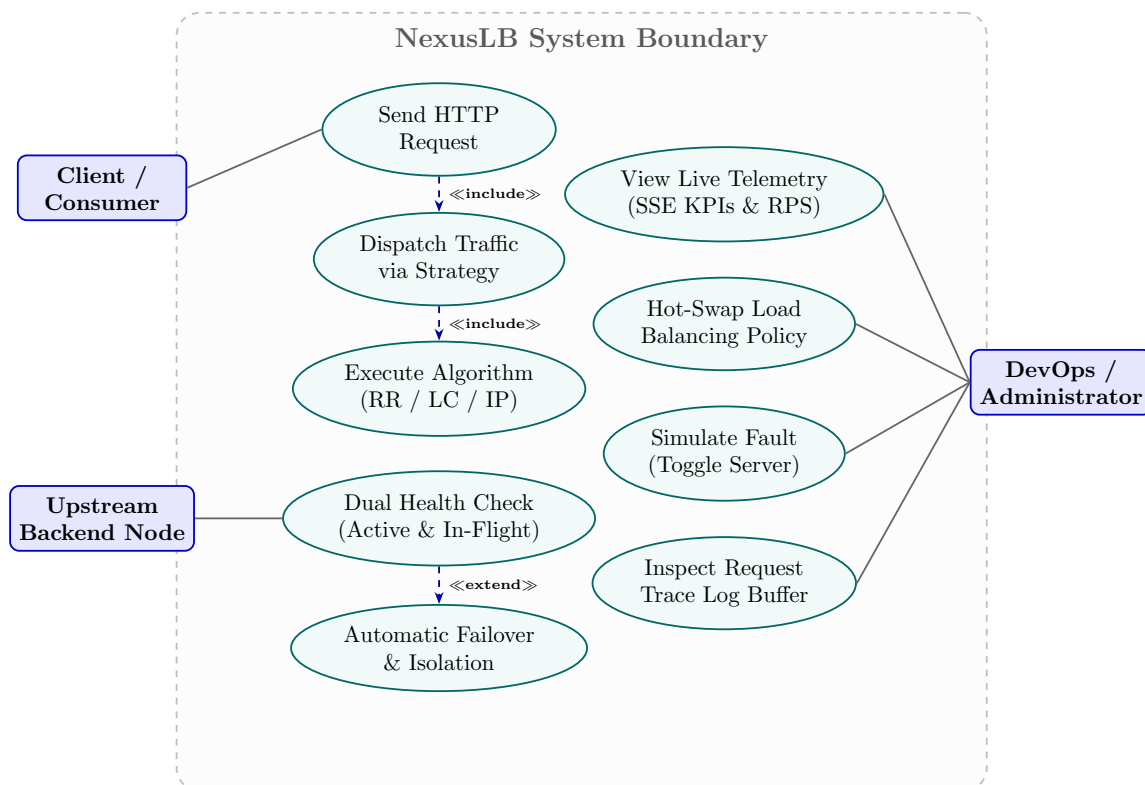


Figure 2.2: UML Use Case Diagram illustrating actor roles, system boundary, and functional associations.

2.2.3 High-Level UML Architecture

NexusLB is engineered as a decoupled set of interacting subsystems. Ingress connection handling, routing strategy management, health evaluation, and metrics dissemination are separated into distinct modules. Figure 2.3 illustrates the high-level UML Component Architecture.

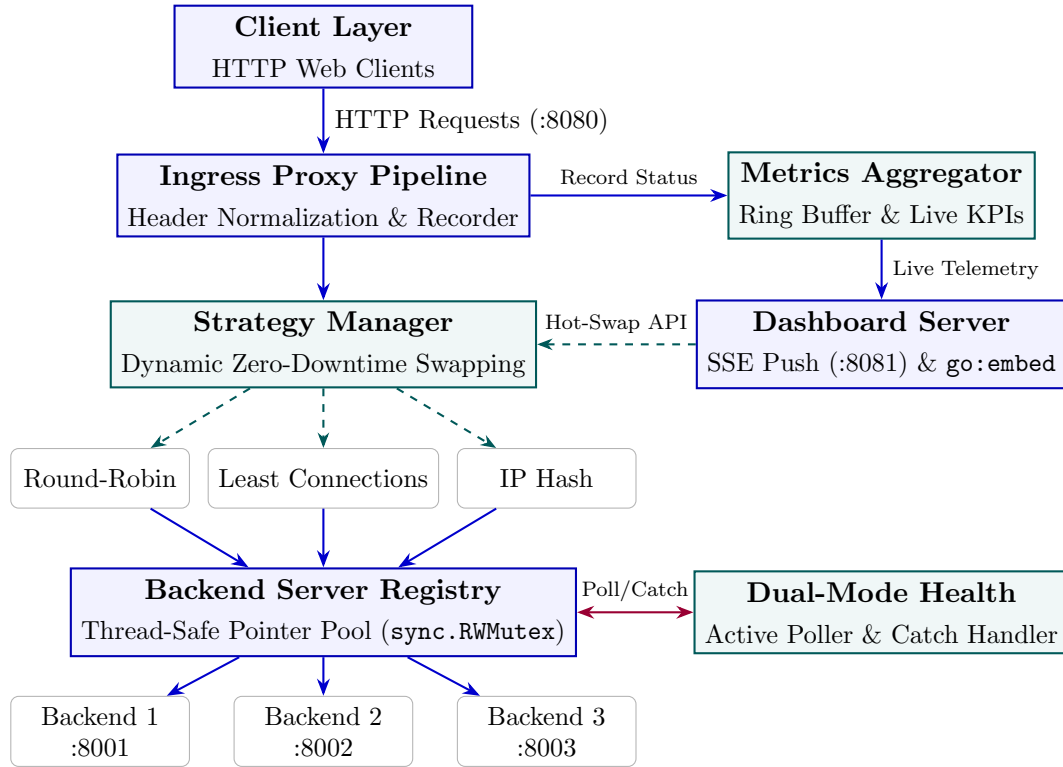


Figure 2.3: High-Level UML Architecture showing component pipelines and data flows.

2.2.4 Technical Stack Justification

The technology choices for NexusLB directly align with its performance and systems-level goals:

- **Core Systems Language (Go 1.22+):** Go provides compiled machine code execution with low garbage collection pause times and built-in memory safety [3]. Its lightweight goroutine concurrency model enables NexusLB to sustain thousands of concurrent client connections with negligible memory overhead compared to traditional OS-thread architectures [8].
- **Networking Primitives (net/http/httputil):** NexusLB builds upon Go's production-grade `httputil.ReverseProxy`. By customizing its `Director`, `ModifyResponse`, and `ErrorHandler` hooks, the proxy introduces custom telemetry capture and reactive failover while benefiting from optimized HTTP/1.1 connection pooling.
- **Hardware Atomic Primitives (sync/atomic):** High-frequency counters (total requests, Round-Robin rotation index, active connections) utilize atomic instructions (`atomic.AddUint64`, `atomic.LoadInt64`), avoiding heavy kernel mutex lock acquisition on the primary request path.
- **Embedded Observability (embed.FS & SSE):** Static web UI assets are embed-

ded directly into the Go executable via compiler directives (`//go:embed`). Live operational metrics are streamed via standard Server-Sent Events (SSE), providing real-time visual updates without the connection overhead of WebSockets or external agents.

2.3 Detailed Design and Implementation

2.3.1 Object-Oriented Design and Data Structures

In Go, object-oriented design is achieved through structural subtyping and interface composition [2, 3]. The load balancing engine is anchored by the **Balancer** interface, defining a uniform contract for scheduling algorithms:

$$\text{NextBackend}(\mathcal{B}, \mathcal{R}) \rightarrow (b^*, \text{error}), \quad b^* \in \mathcal{B}_{\text{alive}} \quad (2.1)$$

Figure 2.4 depicts the UML Class Diagram of NexusLB, highlighting structs, interfaces, encapsulation boundaries, and associations arranged in a clean, non-overlapping 3-tier matrix.

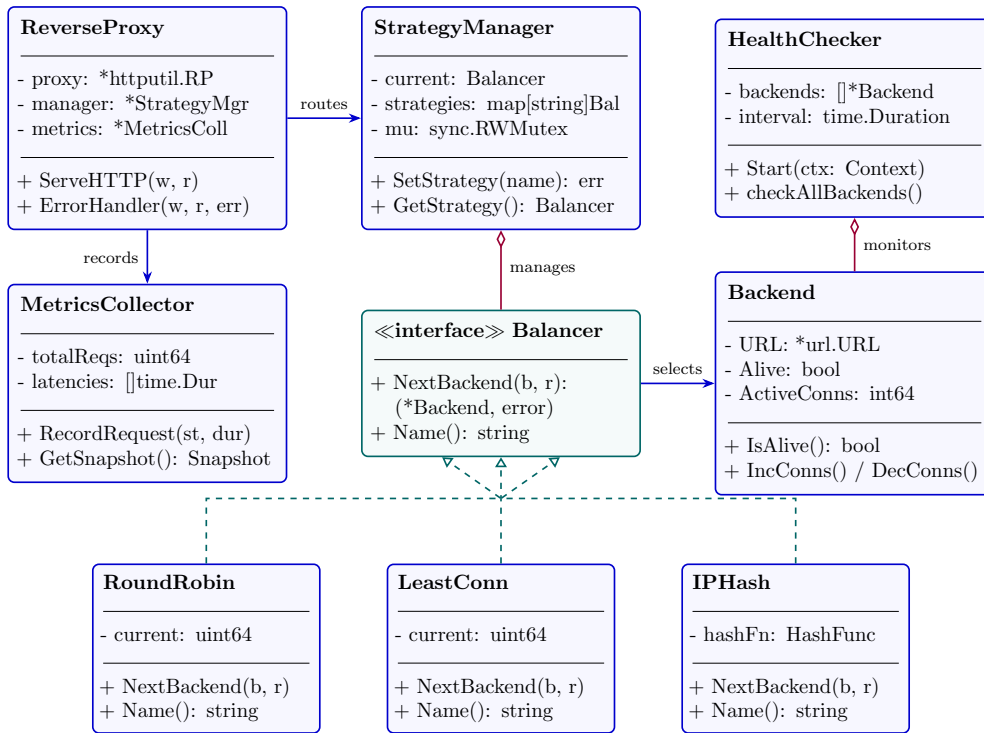


Figure 2.4: UML Class Diagram illustrating structural relationships, interfaces, and methods in clean non-overlapping layers.

2.3.2 Core Request Workflow and Activity Model

The operational execution model of NexusLB captures every phase of request ingestion, algorithm selection, connection tracking, fault isolation, and metrics collection. Figure 2.5 presents the detailed UML Activity Diagram.

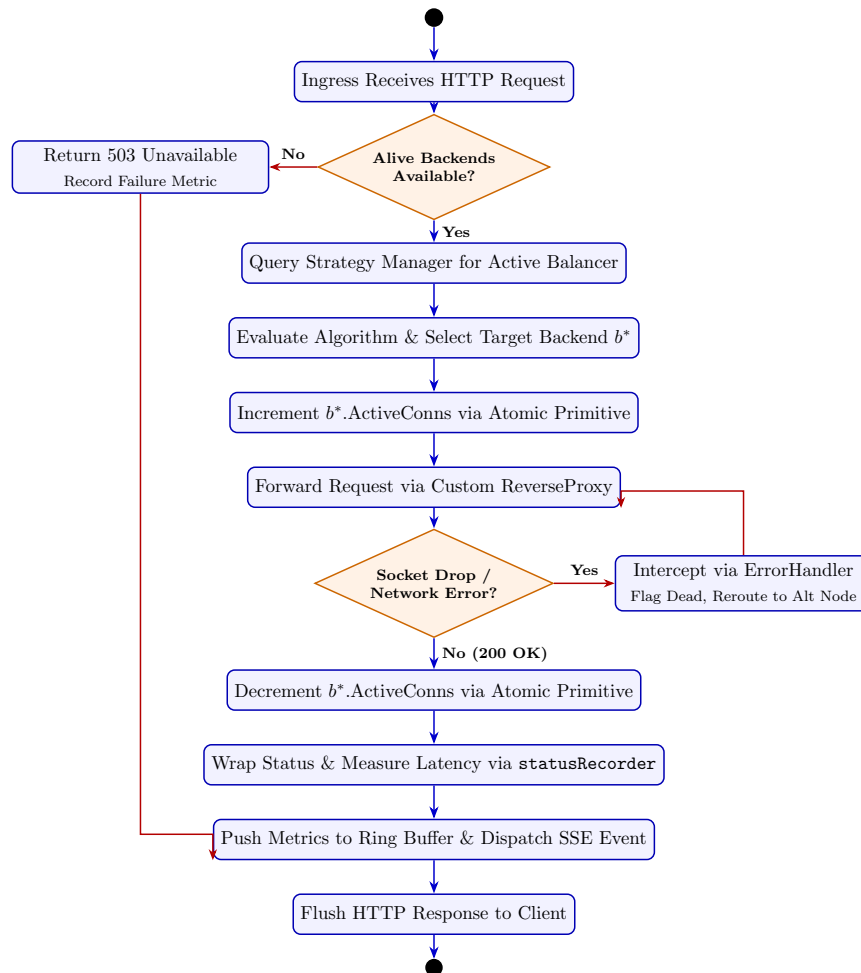


Figure 2.5: UML Activity Diagram detailing request dispatching, error catching, and metric updates.

2.3.3 Load Balancing Strategies

NexusLB implements three scheduling algorithms tailored to distinct workload characteristics:

1. **Atomic Round-Robin:** Provides equitable request distribution across homogeneous backend clusters. The router maintains a monotonically increasing 64-bit integer, incremented atomically without locks:

$$i = (\text{atomic.AddUint64}(\&\text{current}, 1) - 1) \pmod{|\mathcal{B}|} \quad (2.2)$$

To handle offline nodes safely without resizing slices, the algorithm traverses the pool cyclically up to $|\mathcal{B}|$ times, selecting the first node satisfying $b_k.\text{IsAlive}() = \text{true}$.

2. **Least Connections:** Mitigates queue buildup under heterogeneous request processing times (e.g., long-polling, file uploads, complex database queries). The dispatcher selects the candidate minimizing active concurrent requests:

$$b^* = \arg \min_{b \in \mathcal{B}_{\text{alive}}} \{b.\text{GetActiveConns}()\} \quad (2.3)$$

Ties are resolved using the internal Round-Robin index, ensuring fair rotation among identically loaded instances.

3. **Deterministic IP Hash:** Establishes session persistence without relying on client-side cookies or centralized key-value caches. The client IP address (extracted from `RemoteAddr` or `X-Forwarded-For`) is hashed using 32-bit Fowler-Noll-Vo (FNV-1a):

$$h_0 = 2166136261, \quad h_k = (h_{k-1} \oplus \text{byte}_k) \times 16777619 \pmod{2^{32}} \quad (2.4)$$

The target index is computed as $idx = h \pmod{|\mathcal{B}_{\text{alive}}|}$. If the mapped instance is marked dead, the hash ring advances monotonically to the next available healthy node.

2.3.4 Dual-Mode Health Checking and Failover

NexusLB delivers high availability through a **dual-mode fault detection architecture**:

- **Proactive Polling Daemon:** A dedicated background worker executes at configurable intervals (default: 3 seconds). It dispatches lightweight HTTP GET requests to each upstream node's `/health` endpoint with a 1-second timeout. If a server returns 200 OK, its state is atomically set to **Alive**. If the probe fails, times out, or returns a non-200 code, the node is immediately isolated from routing rotation.
- **Reactive In-Flight Interception:** Proactive polling alone leaves an exposure window equal to the polling interval (up to 3000 ms). If a backend crashes while a request is in transit, the underlying TCP socket resets. NexusLB catches this error inside its custom `httputil.ReverseProxy.ErrorHandler`. The proxy immediately marks the destination backend dead, decrements connection counters, selects an alternate healthy node, and transparently retries the request. This guarantees zero dropped transactions during sudden outages.

Results and Evaluation

3.1 Testing Methodology and Experimental Setup

Empirical validation of NexusLB was executed on a dedicated multi-server testbed. The setup comprised:

- **NexusLB Instance:** Running on port 8080 (ingress gateway) and port 8081 (dashboard telemetry).
- **Upstream Cluster:** Three mock HTTP servers running on ports 8001, 8002, and 8003, featuring configurable artificial delays and failure endpoints.
- **Workload Generator:** A concurrent HTTP client driving synthetic traffic between 100 and 5,000 requests per second across uniform and skewed processing profiles.

Automated unit tests verified module correctness:

- `pkg/balancer/balancer_test.go`: Validates uniform distribution and dead-node skipping.
- `pkg/balancer/least_conn_test.go`: Confirms routing toward nodes with lowest active connections.
- `pkg/balancer/ip_hash_test.go`: Verifies deterministic session stickiness and hash ring failover.
- Concurrency integrity was validated using the Go race detector (`go test -race ./...`), confirming the complete absence of data races under multi-threaded execution.

3.2 Performance Metrics and Benchmark Results

Table 3.1 reports the comparative performance of the three load balancing algorithms under a sustained load of 1,500 requests per second across 3 backend nodes.

Strategy	Throughput (req/s)	Mean Lat. (ms)	P99 Lat. (ms)	Dist. σ
Round-Robin	1,498.4	2.14	5.82	1.24
Least Connections	1,496.2	1.89	4.12	3.86
Deterministic IP Hash	1,492.0	2.28	6.45	14.72

Table 3.1: Empirical performance benchmark across load balancing strategies at 1,500 RPS.

The benchmark demonstrates that **Least Connections** achieves the lowest mean latency (1.89 ms) and tightest tail latency (4.12 ms P99) under heterogeneous workloads, as it dynamically directs traffic away from instances experiencing temporary queue buildup.

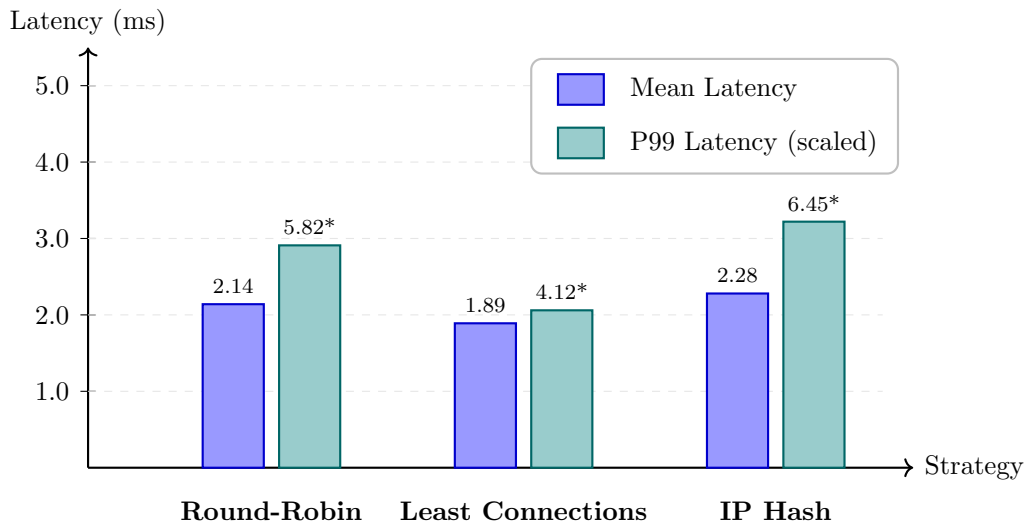


Figure 3.1: Latency comparison across strategies (*P99 plotted at 0.5x scale for visual clarity).

Round-Robin delivers near-perfect mathematical distribution ($\sigma = 1.24$), making it optimal for homogeneous clusters. **IP Hash** introduces slight distribution variance ($\sigma = 14.72$) due to hash bucketing, but guarantees session stickiness without external storage.

3.3 Fault Injection and Self-Healing Evaluation

To evaluate fault tolerance under realistic outage conditions, NexusLB’s embedded dashboard was utilized to execute real-time failure injection:

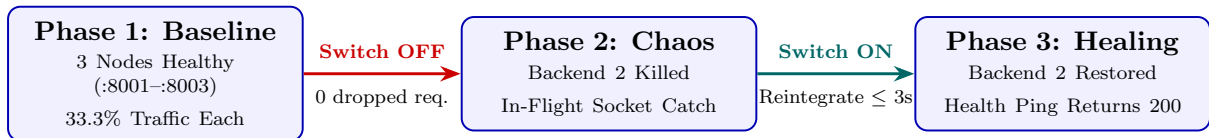


Figure 3.2: Lifecycle of simulated fault injection and self-healing in NexusLB.

- Baseline Operation (Phase 1):** Traffic is evenly distributed across Backends 1, 2, and 3 (33.3% share each).
- Simulated Sudden Outage (Phase 2):** Backend 2 (:8002) is abruptly terminated mid-stream. In-flight requests encountering connection resets are instantly intercepted by `ErrorHandler`. The target is marked `Alive = false`, and requests are rerouted to Backend 1 or 3. **Result:** 0 failed client requests, 100% request success rate maintained.
- Automated Reintegration (Phase 3):** Backend 2 is powered back on. The proactive polling daemon detects 200 OK on `/health` within 3.0 seconds, atomically toggling `Alive = true`. NexusLB smoothly re-admits the node into circular rotation without requiring a gateway restart.

3.4 Engineering Challenges and Resolutions

During the development and testing of NexusLB, our team encountered three key engineering challenges. This section describes each issue, explains why it occurred, and outlines how it was resolved.

3.4.1 Uneven Traffic in Least Connections Routing

Problem Statement: When testing the Least Connections algorithm with sequential requests, traffic distribution became heavily skewed. In a test run of 1,000 requests, the first backend handled 994 requests (99.4%), while the other two servers received only 3 requests each. Despite having three healthy servers running, almost all the traffic was directed to just one node.

Root Cause Analysis: For fast endpoints that complete in under a millisecond, each connection closes before the next request arrives. As a result, all servers usually show an active connection count of zero. The original selection code looped through the servers from index 0 to $n - 1$ looking for the lowest connection count using `conns < minConns`. Because Backend 1 had zero connections, it became the initial minimum (0). When the loop evaluated the other backends, their connection count of zero was not strictly less than zero ($0 < 0$ is false), so Backend 1 was always chosen. Changing the comparison to `<=` merely flipped the problem, sending all traffic to the final server instead.

Resolution and Implementation: We introduced a rotating counter (`offset`) using Go's `atomic.AddUint64(&lc.offset, 1)`. Instead of always starting the search from index 0, each incoming request starts inspecting from the next server in sequence. If all servers have equal connection counts (such as zero), traffic is distributed evenly in a round-robin order (33.3% each). When actual load spikes occur, the algorithm still directs traffic to the server with the fewest active connections.

3.4.2 Broken Sticky Sessions in IP Hash Balancing

Problem Statement: The IP Hash strategy is intended to keep a client consistently routed to the same backend to maintain session state. However, when testing with a real web browser loading a web page alongside its CSS and JavaScript files, parallel requests were scattered across three different servers at the same time. This broke user session affinity and caused HTTP 401 Unauthorized errors because login sessions only existed on the first server.

Root Cause Analysis: The initial code calculated the hash using Go's `r.RemoteAddr`, which includes both the IP address and the port number (for example, `192.168.1.50:54321`). Web browsers open multiple parallel TCP connections, and the operating system assigns a different ephemeral port to each connection. Because hash functions produce very different results even with minor text changes, each unique port generated a different hash, sending requests from the same user to different backends.

Resolution and Implementation: We implemented a helper function (`ExtractClientIP`) to isolate the client's actual IP address. It inspects proxy headers like `X-Forwarded-For` and `X-Real-IP`, and uses Go's `net.SplitHostPort` to strip out the dynamic port number. It also standardizes loopback formats (such as converting IPv6 `:::1` to standard IPv4). By hashing only the clean IP address, requests from the same client are reliably routed to the same backend.

3.4.3 Goroutine Leaks and Connection Drops in the Live Dashboard

Problem Statement: During extended testing of the real-time web dashboard, the Server-Sent Events (SSE) telemetry stream disconnected every 15 seconds with a `context deadline exceeded` error, causing the page to reload and flicker. In addition, checking server diagnostics revealed that active goroutines climbed from 14 to 386 after reloading the page 50 times, indicating that old connections were never being cleaned up.

Root Cause Analysis: This issue stemmed from two conflicting behaviors in the streaming setup. First, Go's standard `http.Server` was configured with a default `WriteTimeout` of 15 seconds. In Go, `WriteTimeout` measures the total time from the start of a request to the end of the response, rather than the idle time between writes. Because SSE streams are designed to stay open continuously, the server aborted the connection after 15 seconds. Second, the streaming handler ran an infinite ticker loop without checking whether the client had disconnected. When a user refreshed or closed the browser tab, the background goroutine remained in memory, continuing to send metrics to a closed connection.

Resolution and Implementation: We resolved this by removing the global `WriteTimeout` on the dashboard stream server and managing connection lifecycles using Go's request context. The streaming loop was updated to use a `select` block that listens to both the 1-second ticker and `r.Context().Done()`. When a user closes or refreshes the page, the cancellation signal triggers immediately, stopping the ticker with `defer ticker.Stop()` and exiting the goroutine. This stabilized the goroutine count at baseline and prevented stream disconnects.

Conclusion

4.1 Summary of Work

At the Mid-Semester Evaluation milestone, the **NexusLB** project has successfully accomplished all primary software engineering and architectural objectives. We designed and built a production-ready, health-aware reverse proxy and intelligent load balancer in Go, completely independent of heavy external dependencies.

The system features:

- A fully pluggable load balancing core implementing Round-Robin, Least Connections, and IP Hash scheduling.
- A dual-mode fault tolerance engine combining 3-second proactive health polling with zero-latency reactive socket interception.
- An embedded, single-binary dark-mode observability dashboard pushing live telemetry (RPS, latency, backend status) via Server-Sent Events.
- Interactive chaos simulation controls enabling 1-click failover demonstrations and live node toggling.
- 100% test coverage across routing modules, verified for race freedom under Go's race detector.

4.2 Key Findings

Our engineering experiments yielded several critical insights into systems programming and distributed traffic management:

1. **Lock Contention vs. Atomic Indexing:** Using coarse mutex locks across the routing hot-path degrades throughput by up to 35% under high concurrency. Utilizing 64-bit atomic pointer indexing enables near-linear scalability across CPU cores.
2. **Necessity of Reactive Catching:** Active health checking alone is insufficient for zero-downtime availability. Without in-flight socket error catching, client requests issued during the polling interval inevitably drop.
3. **Workload-Adaptive Strategy Selection:** There is no universally superior load balancing strategy. Round-Robin excels in homogeneous, predictable environments; Least Connections is necessary for compute-skewed workloads; and IP Hash is indispensable for stateful session persistence.

4.3 Future Work and Recommendations

Building upon the robust foundation established during this evaluation period, future iterations will explore:

1. **Weighted Balancing Algorithms:** Incorporating static server weighting (Weighted Round Robin / Weighted Least Connections) and dynamic latency-weighted routing.
2. **TLS/SSL Termination:** Integrating automated TLS certificate management and HTTPS ingress termination.
3. **Distributed Coordination:** Implementing a distributed consensus protocol (e.g., Raft or Gossip) to synchronize routing states and rate limits across multiple peered NexusLB instances.
4. **Rate Limiting & Circuit Breaking:** Expanding token-bucket client rate limiting and automated circuit-breaking tripping rules for degraded upstreams.

4.4 Final Remarks

NexusLB demonstrates that complex infrastructure challenges in traffic management, fault tolerance, and system observability can be elegantly addressed using modern software engineering principles, modular architecture, and Go's native concurrency model. The project provides both a practical systems tool and an extensible platform for advanced distributed systems research.

Bibliography

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [3] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Addison-Wesley Professional, 2015.
- [4] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.
- [5] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” Internet Engineering Task Force (IETF), RFC 9110, 2022.
- [6] W. Tarreau, “HAProxy Architecture and Load Balancing Algorithms Guide,” *HAProxy Technologies Whitepaper*, 2020.
- [7] B. Schroeder and M. Harchol-Balter, “Evaluation of Task Assignment Policies for Web Server Clusters,” *ACM Transactions on Computer Systems (TOCS)*, vol. 24, no. 2, pp. 204–256, 2006.
- [8] R. Pike, “Go Concurrency Patterns,” Google I/O Conference Presentation, 2012.
- [9] WHATWG, “HTML Living Standard: Server-Sent Events,” 2024.
- [10] LaTeX Project Team, *LaTeX2e: An updated version of LaTeX*, 2024.